

JavaScript Questions and Answers

1. How many ways we can create objects in JavaScript?

1. Object Literal (Most Common)

```
</> JavaScript

const person = {
  name: "Venkat",
  age: 30,
  city: "Bangalore"
};

console.log(person.name);
```

- ✓ Simple and easiest way

2. Using `new Object()`

```
</> JavaScript

const person = new Object();

person.name = "Venkat";
person.age = 30;

console.log(person);
```

- ✓ Built-in object constructor

3. Using Constructor Function

```
</> JavaScript

function Person(name, age) {
  this.name = name;
  this.age = age;
}

const p1 = new Person("Venkat", 30);

console.log(p1);
```

- ✓ Reusable object creation
- ✓ Used before ES6 classes

4. Using ES6 Class

```
</> JavaScript

class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

const p1 = new Person("Venkat", 30);

console.log(p1);
```

- ✓ Modern and recommended approach

5. Using `Object.create()`

```
</> JavaScript

const personPrototype = {
  greet() {
    console.log("Hello");
  }
};

const person = Object.create(personPrototype);

person.name = "Venkat";

console.log(person.name);
person.greet();
```

- ✓ Used for prototype inheritance

6. Using Factory Function

```
</> JavaScript

function createPerson(name, age) {
  return {
    name,
    age
  };
}

const p1 = createPerson("Venkat", 30);

console.log(p1);
```

- ✓ Clean and reusable approach

7. Using Singleton Pattern

```
<> JavaScript

const person = new function () {
  this.name = "Venkat";
  this.age = 30;
};

console.log(person);
```

✓ Creates only one object instance

Interview Answer (Short Version)

"There are several ways to create objects in JavaScript such as:

1. Object Literal
2. new Object()
3. Constructor Function
4. ES6 Class
5. Object.create()
6. Factory Function
7. Singleton Pattern

Most commonly used methods are Object Literal and ES6 Classes."

2. Is there any Singleton pattern to create objects?

7. Using Singleton Pattern

```
<> JavaScript

const person = new function () {
  this.name = "Venkat";
  this.age = 30;
};

console.log(person);
```

✓ Creates only one object instance

3. Var , let and const differences

JavaScript provides three ways to declare variables: var, let, and const, but they differ in scope, hoisting behaviour, and re-assignment rules.

var: Declares variables with function or global scope and allows re-declaration and updates within the same scope.

let: Declares variables with block scope, allowing updates but not re-declaration within the same block.

const: Declares block-scoped variables that cannot be reassigned after their initial assignment.

```
// var example
var x = 10;
var x = 20; // Re-declaration allowed
x = 30; // Update allowed
console.log(x); // Output: 30

// let example
let y = 10;
// let y = 20; // Re-declaration NOT allowed
y = 25; // Update allowed
console.log(y); // Output: 25

// const example
const z = 10;
// z = 20; // Re-assignment NOT allowed
console.log(z); // Output: 10

// Block scope demonstration
if (true) {
  var a = 1;
  let b = 2;
  const c = 3;
}

console.log(a); // Works (var is function/global scoped)
// console.log(b); // Error (let is block scoped)
// console.log(c); // Error (const is block scoped)
```

4. How does scope gets determined in JavaScript

In JavaScript, scope is determined by **where** you write your code (lexical scoping) and **how** you declare your variables (var, let, or const).

JavaScript determines scope mainly based on:

1. Where the variable is declared
2. Which keyword is used (var, let, const)
3. Function blocks and code blocks {}

Types of Scope in JavaScript

1. Global Scope
2. Function Scope
3. Block Scope
4. Lexical Scope

1. Global Scope

Variable declared outside all functions/blocks.

```
<> JavaScript
let company = "HexaCorp";

function show() {
  console.log(company);
}

show();
```

✓ Accessible everywhere

2. Function Scope

Variables declared inside function using `var`.

```
<> JavaScript
function test() {
  var name = "Venkat";
  console.log(name);
}

test();

// console.log(name); ✗ Error
```

✓ Accessible only inside function

3. Block Scope

Variables declared using `let` and `const` inside `{}`.

```
JavaScript
{
  let age = 30;
  const city = "Bangalore";
}

// console.log(age); ❌ Error
// console.log(city); ❌ Error
```

✅ Accessible only inside block

4. Lexical Scope (VERY IMPORTANT)

JavaScript uses Lexical Scope.

That means:

Scope is determined by where functions are written in the code, not where they are called.

Example

```
JavaScript
function outer() {
  let name = "Venkat";

  function inner() {
    console.log(name);
  }

  inner();
}

outer();
```

✅ `inner()` can access parent variable

This is called Lexical Scope



5. Explain the difference between call, apply, bind methods?

In JavaScript, call, apply, and bind are methods used to explicitly set the this context of a function. While they serve a similar purpose, they differ in how they are executed and how they handle arguments

```
var name={
  firstname:"Venkat",
  lastname:"MK"
}
const printname=function(hometown,state){
  console.log(this.firstname + " " + this.lastname + " " + hometown + " " + state )
}
printname.call(name,"Chennai","TN");
printname.apply(name,["Chennai","TN"]);
let printmyName=printname.bind(name,"Chennai","TN");
printmyName();
```

OutPut:

```
Venkat MK Chennai TN
Venkat MK Chennai TN
Venkat MK Chennai TN
```

6. What is function Currying in JavaScript?

Currying is a functional programming technique that transforms a function with multiple arguments into a sequence of nested functions, each taking a **single argument** at a time

```
// Simple Curried Function
function add(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}
console.log(add(10)(20)(30));

/// Function currying by Closures method
let multiply=function(x){
  return function (y){
    console.log(x*y);
  }
}
let multiplybyTwo=multiply(2);
multiplybyTwo(3);
let multiplybyThree=multiply(3);
multiplybyThree(10);

Output:
60
6
30
```

7. What is Closure in Javascript?

Functions along with lexical scope forms a closure.

A **closure** is the combination of a function bundled together (enclosed) with references to its surrounding state (the **lexical environment**). In other words, a closure gives you access to an outer function's scope from an inner function. In JavaScript, closures are created every time a function is created, at function creation time.

```
// Simple Closure
function init(){
  var name="Venkat";
  function displayName(){
    console.log(name)
  }
  displayName();
}
init()
```

8. What is prototype chain in JavaScript and how does it work?

In JavaScript, every object has an internal link to another object called its **prototype**.

When you try to access a property or method on an object, JavaScript first checks the object itself. If it does not find it, it checks the object's prototype. This process continues until the property is found or the prototype chain ends.

Simple Example

```
JavaScript

const person = {
  name: "Venkat",
  greet: function () {
    console.log("Hello");
  }
};

const employee = {
  salary: 50000
};

employee.__proto__ = person;

console.log(employee.salary); // 50000
console.log(employee.name);   // Venkat
employee.greet();             // Hello
```

Explanation

`salary` is available inside `employee`, so JavaScript takes it directly.

`name` is not available inside `employee`, so JavaScript checks its prototype, which is `person`.

`greet()` is also not inside `employee`, so JavaScript finds it in `person`.

Prototype chaining is a mechanism in JavaScript where objects inherit properties and methods from other objects through their prototype. When a property is accessed, JavaScript first looks in the current object. If it is not found, it searches the prototype object, then the prototype's prototype, until it reaches null.

9. what array slice and splice methods in JavaScript?

In JavaScript, `slice()` and `splice()` are array methods with distinct purposes. `slice()` creates a new array containing selected elements from the original, while `splice()` modifies the original array by adding, removing, or replacing elements.

1. slice() Method

`slice()` is used to copy a portion of an array.

Syntax

```
JavaScript
array.slice(start, end)
```

- `start` → starting index
- `end` → ending index (not included)

Example 1

```
JavaScript
let arr = [10, 20, 30, 40, 50];

let result = arr.slice(1, 4);

console.log(result);
```

Output

```
JavaScript
[20, 30, 40]
```

Explanation

- Starts from index `1`
- Ends before index `4`



Example 3 — Negative Index

```
JavaScript
let arr = [10, 20, 30, 40, 50];

console.log(arr.slice(-3));
```

Output:

```
JavaScript
[30, 40, 50]
```

Negative index starts from end.

2. splice() Method

`splice()` is used to:

- Remove elements
- Add elements
- Replace elements

It changes the original array.

Syntax

```
</> JavaScript
array.splice(start, deleteCount, item1, item2)
```

- `start` → index position
- `deleteCount` → how many items remove
- `item1,item2` → items to add

Remove Elements

Example

```
</> JavaScript
let arr = [10, 20, 30, 40, 50];

arr.splice(1, 2);

console.log(arr);
```

Output

```
</> JavaScript
[10, 40, 50]
```

Add Elements

Example

```
</> JavaScript
let arr = [10, 20, 50];

arr.splice(2, 0, 30, 40);

console.log(arr);
```

Output

```
</> JavaScript
[10, 20, 30, 40, 50]
```

Explanation

- Start at index `2`
- Remove `0`
- Add `30` and `40`

Replace Elements

Example

```
</> JavaScript
let arr = [10, 20, 30, 40];

arr.splice(1, 2, 100, 200);

console.log(arr);
```

Output

```
</> JavaScript
[10, 100, 200, 40]
```

Explanation

Removed:

```
</> JavaScript
20, 30
```

Added:

```
</> JavaScript
100, 200
```

Difference Between `slice()` and `splice()`

Feature	<code>slice()</code>	<code>splice()</code>
Modifies original array	✗ No	✓ Yes
Returns removed items	✗ No	✓ Yes
Used for	Copying	Add/Remove/Replace
Easy to remember	"SLICE = Copy"	"SPICE = Change"

Interview Question

Q: Difference between slice and splice?

Answer

- `slice()` returns a selected portion of an array without changing the original array.
- `splice()` adds, removes, or replaces elements and modifies the original array.

10. Explain first class functions in JavaScript?

The ability of functions to be used as values and can be pass this in an argument to other functions and can be returned from the functions. Is this ability known as first class functions

A first-class function means functions are treated as values in JavaScript. Functions can be assigned to variables, passed as arguments, returned from other functions, and stored in objects or arrays.

```
//First Class functions example 1
var b = function (params){
    console.log(params);
}
b(function(){
})
Output
[Function (anonymous)]

// if I pass the anonymous function as a params of another function
```

```
//First Class functions example 2
var b = function (params){
    console.log(params);
}
function xyz(){
}
b(xyz);

Output:
[Function: xyz]
// if I passing the another function inside a function
```

```
//First Class functions example 3
var b = function (params){
    return function xyz(){
    }
}
console.log(b());

output:
[Function: xyz]

// Return the named of anonyms function from function when call the function
```

11. Difference Function Statement and Function Expression

A Function Statement (Function Declaration) defines a function directly using the function keyword and is fully hoisted.

A Function Expression stores a function inside a variable. Only the variable is hoisted, not the function implementation, so it cannot be called before initialization.

1. Function Statement / Declaration

Syntax

```
<> JavaScript

function greet() {
    console.log("Hello");
}
```

2. Function Expression

Function stored inside variable.

Syntax

```
</> JavaScript

const greet = function () {
  console.log("Hello");
};
```

Hoisting Difference

Function Statement

```
</> JavaScript

test();

function test() {
  console.log("Working");
}
```

✔ Works

Function Expression

```
</> JavaScript

test();

const test = function () {
  console.log("Working");
};
```

✖ Error

Cannot access 'test' before initialization

Because only variable is hoisted, not function value.

12. Difference between let and var in JavaScript?

var is function-scoped and allows redeclaration, while **let** is block-scoped and does not allow redeclaration. Both are hoisted, but **let** stays in the Temporal Dead Zone until initialization, whereas **var** is initialized with undefined.

Difference Table

Feature	var	let
Scope	Function Scope	Block Scope
Redeclaration	✔ Allowed	✖ Not Allowed
Reassignment	✔ Allowed	✔ Allowed
Hoisting	✔ Hoisted with undefined	✔ Hoisted but in TDZ
Temporal Dead Zone	✖ No	✔ Yes
Global Object Attachment	✔ Yes	✖ No
Introduced In	Old JavaScript	ES6 (Modern JS)

13. Explain JavaScript execution

JavaScript is Synchronous and Single Threaded language

JavaScript can execute one command at a time with specific order so that means that it can only go the next line once the current line has been finished executing. So that means Synchronous and Single Threaded language

When JavaScript code runs, the JavaScript engine executes it in a specific process.

Mainly JavaScript execution happens in **2 phases**:

1. **Memory Creation Phase**
2. **Code Execution Phase**

This happens inside something called the **Execution Context**.

JavaScript Execution Flow

```
JavaScript Code
↓
Execution Context Created
↓
Memory Creation Phase
↓
Code Execution Phase
```

Example

```
<> JavaScript
var a = 10;
var b = 20;

function add(x, y) {
  return x + y;
}

var result = add(a, b);

console.log(result);
```

Phase 1 — Memory Creation Phase

JavaScript scans the code first.

It allocates memory for:

- Variables
- Functions

Memory Allocation

```
a    → undefined
b    → undefined
result → undefined
add  → function definition
```

Functions are fully stored in memory.

Variables get `undefined`.

Phase 2 — Code Execution Phase

Now JavaScript executes line by line.

Step 1

```
<> JavaScript
var a = 10;
```

Memory updates:

```
a → 10
```

Step 2

```
<> JavaScript
var b = 20;
```

Memory updates:

```
b → 20
```

Step 3

Function already exists in memory.

No execution now.

Step 4

```
<> JavaScript
var result = add(a, b);
```

Function call happens.

A new **Function Execution Context** is created.

14. What is IIFE and why it is used in javascript

What is IIFE in JavaScript?

IIFE stands for:

Immediately Invoked Function Expression

It is a function that:

1. Is created immediately
2. Executes immediately after creation

Basic Syntax

```
</> JavaScript  
  
(function () {  
    console.log("Hello");  
})();
```

Output

Hello

15. What is Shallow Copy and Deep Copy

When copying objects or arrays in JavaScript, there are two types of copying:

1. Shallow Copy
2. Deep Copy

Shallow Copy

A shallow copy copies only the first level.

If the object contains nested objects or arrays, the references are copied instead of creating new independent objects.

Deep Copy

Deep copy creates completely independent copies including nested objects.

No shared references.

Example of Shallow Copy

```
const obj1 = {
  name: "Venkat",
  address: {
    city: "Bangalore"
  }
};

const obj2 = {...obj1};

obj2.name="Kumar";
obj2.address.city = "Chennai";

console.log(obj1);
console.log(obj2);
```

Output:

```
{ name: 'Venkat', address: { city: 'Chennai' } }
{ name: 'Kumar', address: { city: 'Chennai' } }
```

Example of Deep Copy

```
const obj1 = {
  name: "Venkat",
  address: {
    city: "Bangalore"
  }
};

//const obj2 = structuredClone(obj1);
const obj2 = JSON.parse(JSON.stringify(obj1));
obj2.name="Kumar";
obj2.address.city = "Chennai";

console.log(obj1);
console.log(obj2);
```

OutPut:

```
{ name: 'Venkat', address: { city: 'Bangalore' } }
{ name: 'Kumar', address: { city: 'Chennai' } }
```

16. What is .map(), .reduce(), .filter()?

.map()- allows you to iterate over an array and modify its elements using a callback function, creating a new array with modified elements

```
// Interview Questions

// 1. Create a list of employees and change their names
// in to Lowercase.

const employees = ['Hari', 'Murugan', 'Aadhiran', 'Jai',
'Vel', 'John']

const usingMap = employees.map(employee => employee.
toLowerCase() )
console.log('originalArray', employees)
console.log('usingMap', usingMap);
```

JavaScript .map (🔗).reduce (🔗).filter (🔗)

.map() - allows you to iterate over an array and modify its elements using a callback function, creating a new array with the modified elements

.filter() - Filter method creates a new array by filtering elements based on a specified condition

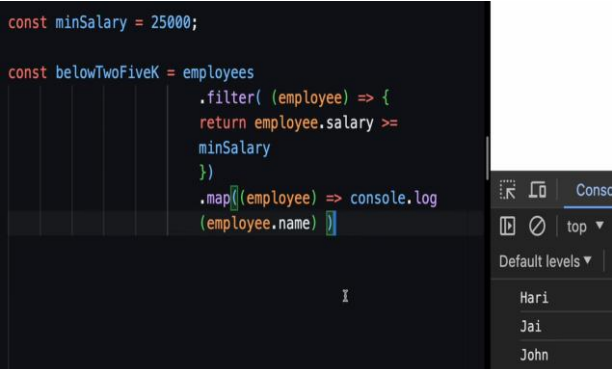
```
// Interview Questions

// 2. Create an array of employee objects with details
such as name, age, years of experience, and salary.
Then, filter the employees whose salary is above 25k and
display their names.

const employees = [
  {id: 1, name: 'Hari', age:25, experience: 3, salary:
  30000 },
  {id: 2, name: 'murugan', age:43, experience: 13,
  salary: 20000 },
  {id: 3, name: 'Aadhiran', age:23, experience: 1,
  salary: 18000 },
  {id: 4, name: 'Jai', age:28, experience: 5, salary:
  32500 },
  {id: 5, name: 'Vel', age:27, experience: 3, salary:
  12000 },
  {id: 6, name: 'John', age:32, experience: 8, salary:
  40000 },
]

const minSalary = 25000;

const belowTwoFiveK = employees
  .filter( (employee) => {
    return employee.salary >=
    minSalary
  })
  .map((employee) => console.log
    (employee.name) )
```



.reduce() - reduce method reduces an array to single value by applying a function to each element

```
// 3. How to calculate all employee salary?

const employees = [
  {id: 1, name: 'Hari', age:25, experience: 3, salary:
  30000 },
  {id: 2, name: 'murugan', age:43, experience: 13,
  salary: 20000 },
  {id: 3, name: 'Aadhiran', age:23, experience: 1,
  salary: 18000 },
  {id: 4, name: 'Jai', age:28, experience: 5, salary:
  32500 },
  {id: 5, name: 'Vel', age:27, experience: 3, salary:
  12000 },
  {id: 6, name: 'John', age:32, experience: 8, salary:
  40000 },
]

16
17   const totalSalary = employees.reduce((accumulator,
18   employee ) => {
19     return accumulator + employee.salary
20   },0 )
21   console.log('Total Salary', totalSalary);
22
23
24
25
```

17. What is call back function?

A **callback** function in JavaScript is a function that is passed as an argument to another function and is executed after the completion of that function.

Callbacks are commonly used in asynchronous operations, such as handling events, making network requests, or working with timers

```
// callback function example
function uploadVideo(callback){
  console.log("Uploading video");
  setTimeout(()=>{
    console.log("Uploading Done");
    callback();
  },3000);
}

function pulishVideo(publishcallback){
  console.log("Publishing video");
  setTimeout(()=>{
    console.log("Publishing Done");
    publishcallback();
  },2000)
}

function notifysubscribers(){
  console.log("Notification Done");
}

uploadVideo(function(){
  pulishVideo(function(){
    notifysubscribers();
  })
})
```

```
Uploading video
Uploading Done
Publishing video
Publishing Done
Notification Done
```

```
=== Code Execution Successful ===
```

Another callback example

```
// callback function example with data passing
function uploadVideo(videodata,callback){
  console.log("Uploading video");
  setTimeout(()=>{
    console.log("Uploading Done" + videodata.title);
    callback(videodata);
  },3000);
}

function pulishVideo(videodata,publishcallback){
  console.log("Publishing video");
  setTimeout(()=>{
    console.log("Publishing Done -- " + videodata.title );
    publishcallback(videodata);
  },2000)
}

function notifysubscribers(videodata){
  console.log("Notification Sent -- " + videodata.title);
}

const data={
  title:"This is new callback video demo",
  description:"some desc about call back"
}

uploadVideo(data,function(uploadedData){
  pulishVideo(uploadedData,function(publishedData){
    notifysubscribers(publishedData);
  })
})
```

```
Uploading video
Uploading DoneThis is new callback video demo
Publishing video
Publishing Done -- This is new callback video demo
Notification Sent -- This is new callback video demo
```

```
=== Code Execution Successful ===
```

Callback vs Promise vs Async

```
#CALLBACK
getUser(function(err, user){
  GetProfile(user, function(err, profile){
    GetAccount(profile, function(err, acc){
      GetReport(acc, function(err, report){
        SendStatistics(report, function(e){
          ...
        });
      });
    });
  });
});

#ASYNC/AWAIT
async function SendAsync() {
  let user = await GetUser(1)
  let profile = await GetProfile(user);
  let account = await GetAccount(profile);
  let report = await GetReport(account);

  let send = SendStatistic(report);

  console.log(send)
}

#PROMISE
getUser()
  .then(GetProfile)
  .then(GetAccount)
  .then(GetReport)
  .then(SendStatistics)
  .then(function (success) {
    console.log(success)
  })
  .catch(function (e) {
    console.error(e)
  })
})
```

18. Explain about the Promises

Promises are a fundamental part of modern JavaScript and are widely used for managing asynchronous code and handling operations that don't produce immediate results. They improve code readability, maintainability, and error handling in asynchronous workflows.

```
// Promise with example
const data={
  title:"This is Promise function demo",
  description:"some desc about call back"
}

function uploadVideo(){
  // Promise constructor
  return new Promise((resolve,reject)=>{ // resolve,reject is callback
    console.log("Uploading video");
    const isUploaded=true;
    setTimeout(()=>{
      if (isUploaded){
        resolve("Upload Done");
      }else{
        reject("Upload failed");
      }
    },3000);
  });
}

function pulishVideo(){
  // Promise constructor
  return new Promise((resolve,reject)=>{ // resolve,reject is callback
    console.log("Publishing video");
    const isPublished=true;
    setTimeout(()=>{
      if (isPublished){
        resolve("Published Done");
      }else{
        reject("Published failed");
      }
    },2000)
  });
}

function notifysubscribers(videodata){
  console.log("Notification Sent -- "+ videodata.title);
}

// then catch used
uploadVideo().then(isUploaded=>{
  console.log(isUploaded)
  return pulishVideo()
}).then(isPublished=>{
  console.log(isPublished);
  notifysubscribers(data);
}).catch(err =>{
  console.log(err);
})

// async and await used
async function uploadingVideofun(){
  try {
    const isUploaded= await uploadVideo();
    console.log(isUploaded);
    const isPublished=await pulishVideo();
    console.log(isPublished);
    notifysubscribers(data)
  } catch (e) {
  }
}

uploadingVideofun();
```

Output:

Uploading video
Upload Done
Publishing video
Published Done
Notification Sent -- This is Promise function demo

19. Explain Promise.race() , Promise.all() Promise.any()

Promise.all() waits for ALL promises to complete successfully.

If any one promise fails, the entire operation fails.

Waits until ALL promises finish.

Returns results as array.

```
// Promise all
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, 'foo');
});

Promise.all([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});
// expected output: Array [3, 42, "foo"]
```

Real-Time Example

Loading multiple APIs together:

JavaScript

```
Promise.all([
  fetchUsers(),
  fetchOrders(),
  fetchProducts()
]);
```

Used when ALL data required.

Promise.allSettled() waits for ALL promises to complete, whether they succeed or fail.

Unlike Promise.all():

- It does NOT fail if one promise rejects.
- It returns the status of every promise.

Example

```
</> JavaScript

const p1 = Promise.resolve("Success 1");

const p2 = Promise.reject("Error 1");

const p3 = Promise.resolve("Success 2");

Promise.allSettled([p1, p2, p3])
  .then((results) => {
    console.log(results);
  });
```

Output

```
</> JavaScript

[
  { status: "fulfilled", value: "Success 1" },
  { status: "rejected", reason: "Error 1" },
  { status: "fulfilled", value: "Success 2" }
]
```

Promise.allSettled() waits for ALL promises.

For each promise it returns:

Promise.any() returns the FIRST SUCCESSFUL promise.

It ignores failed promises.

Example

```
</> JavaScript

const p1 = Promise.reject("Error 1");

const p2 = new Promise((resolve) => {
  setTimeout(() => resolve("Success"), 1000);
});

const p3 = Promise.reject("Error 2");

Promise.any([p1, p2, p3])
  .then(console.log)
  .catch(console.log);
```

Output

Success

Important Point

`Promise.any()` ignores rejected promises.

Only cares about first success.

Promise.race() returns the FIRST completed promise.

It does not wait for others.

Example

```
<> JavaScript

const p1 = new Promise((resolve) => {
  setTimeout(() => resolve("First"), 2000);
});

const p2 = new Promise((resolve) => {
  setTimeout(() => resolve("Second"), 1000);
});

Promise.race([p1, p2])
  .then(console.log);
```

Output

Second

Because `p2` completed first.

Example with Reject

```
<> JavaScript

const p1 = new Promise((resolve) => {
  setTimeout(() => resolve("Success"), 2000);
});

const p2 = new Promise(, reject) => {
  setTimeout(() => reject("Failed"), 1000);
});

Promise.race([p1, p2])
  .then(console.log)
  .catch(console.log);
```

Output

Failed

Because rejection happened first.

Important Point

First completed promise can be:

- Success
- OR
- Failure

20. How to chain multiple promise

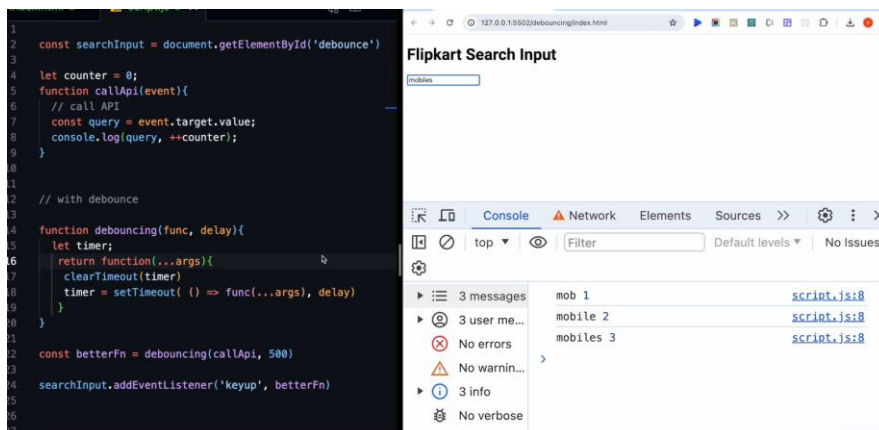
21. What is debouncing?

Debouncing

Debouncing is a programming technique that helps to improve the performance of web applications by limiting the frequency of function calls

Real life examples of debouncing

1. Search Box
2. Submit Button



22. What is DOM API and explain about it

DOM API is a set of methods and properties provided by the browser to interact with HTML and XML documents using JavaScript.

It is an API that used to interact and Manged the HTML documents

Using DOM API, JavaScript can:

- Access HTML elements
- Change content
- Change styles
- Handle events
- Create/remove elements
- Modify attributes

DOM API allows JavaScript to manipulate web page elements dynamically.

DOM API Example

```
<> HTML
<h1 id="title">Hello</h1>
```

```
<> JavaScript
const element = document.getElementById("title");
element.innerText = "Welcome";
```

Output

```
<> HTML
<h1>Welcome</h1>
```

Important DOM Objects

Object	Purpose
document	Represents webpage
window	Represents browser window
element	Represents HTML element

Common DOM API Methods

1. getElementById()
2. getElementsByClassName()
3. getElementsByTagName()
4. querySelector()
5. querySelectorAll()

23. Session Storage, local storage and cookies when to use and when which

Feature	Cookies	localStorage	sessionStorage
Size Limit	~ 4KB	~ 5MB	~ 5MB
Expiry	Customizable	Never	Tab close
Server Access	✔ Yes	✗ No	✗ No
Accessibility	Any window	Any window	Same tab only
Auto-sent	✔ Yes	✗ No	✗ No

1. Local Storage

Local storage stores data permanently until manually removed.

Data remains even after:

- **Browser refresh**
- **Browser close**
- **System restart**

Store Data :

```
localStorage.setItem("username", "Venkat");
```

Get Data

```
const user = localStorage.getItem("username");
```

```
console.log(user);
```

Remove Data

```
localStorage.removeItem("username");
```

Clear All

```
localStorage.clear();
```

2. Session Storage

Session storage stores data only for current browser tab/session.

Data removed when:

- Tab closed
- Browser tab refreshed? ✗ No (still exists)
- Browser/tab completely closed? ✔ Yes

Store Data

```
sessionStorage.setItem("token", "12345");
```

Get Data

```
const token = sessionStorage.getItem("token");
```

```
console.log(token);
```

Remove Data

```
sessionStorage.removeItem("token");
```

Useful for:

Multi-step forms

Temporary navigation state

Session-specific data

3. Cookies

Cookies store small data and are automatically sent to server with every HTTP request.

Create Cookie

```
document.cookie = "username=Venkat";
```

Read Cookie

```
console.log(document.cookie);
```

Set Expiry

```
document.cookie = "username=Venkat; expires=Fri, 31 Dec 2026 12:00:00 UTC";
```

Delete Cookie

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC";
```

Cookies mainly used for:

- Authentication
- Session management
- Tracking
- User preferences

24. What are E6 Features

ES6 (ECMAScript 2015) introduced many important modern JavaScript features.

ES6 made JavaScript:

- Cleaner
- More powerful
- Easier to write and maintain

Most Important ES6 Features

- let and const

- Arrow Functions
- Template Literals
- Destructuring
- Spread Operator
- Rest Parameters
- Default Parameters
- Classes
- Modules
- Promises
- Enhanced Object Literals
- For...of Loop
- Map and Set
- Symbol
- Generators

25. Difference Between normal function and arrow function What about **this** key word How does it behave in both?

1. Syntax Difference

Normal Function

```
</> JavaScript
function add(a, b) {
  return a + b;
}
```

Arrow Function

```
</> JavaScript
const add = (a, b) => {
  return a + b;
};
```

2. **this** Keyword Difference (MOST IMPORTANT)

Normal Function → Dynamic **this**

this depends on how function is called.

Example

```
</> JavaScript
const user = {
  name: "Venkat",

  normalFunction: function () {
    console.log(this.name);
  }
};

user.normalFunction();
```

Output:



Venkat

Arrow Function → Lexical `this`

Arrow function does NOT create its own `this`.

It takes `this` from parent scope.

Example

```
</> JavaScript
const user = {
  name: "Venkat",
  arrowFunction: () => {
    console.log(this.name);
  }
};

user.arrowFunction();
```

Output:

undefined

Because arrow function uses outer/global `this`.

Real-Time Example

Problem with Normal Function

```
</> JavaScript
function Timer() {
  this.seconds = 0;

  setInterval(function () {
    this.seconds++;
    console.log(this.seconds);
  }, 1000);
}

new Timer();
```

✗ `this` refers to window object.

Solution Using Arrow Function

```
</> JavaScript
function Timer() {
  this.seconds = 0;

  setInterval(() => {
    this.seconds++;
    console.log(this.seconds);
  }, 1000);
}

new Timer();
```

✓ Arrow function inherits `this` from Timer.

26. Rest operator Can we use it an array? se case of Object and Array Destructuring

The **Rest Operator (...)** can be used with arrays in JavaScript.

It is used to collect remaining elements into a new array.

Rest Operator in Array Destructuring

Syntax

JavaScript

```
const [a, ...rest] = array;
```

- `a` → first element
- `rest` → remaining elements as array

Example 1

JavaScript

```
const numbers = [10, 20, 30, 40, 50];

const [first, ...remaining] = numbers;

console.log(first);
console.log(remaining);
```

Output

JavaScript

```
10
[20, 30, 40, 50]
```

Explanation

JavaScript

```
first = 10
remaining = [20,30,40,50]
```

Rest operator collects remaining values.

Example 2

JavaScript

```
const colors = ["Red", "Blue", "Green", "Yellow"];

const [c1, c2, ...others] = colors;

console.log(c1);
console.log(c2);
console.log(others);
```

Output

JavaScript

```
Red
Blue
["Green", "Yellow"]
```

Important Rule

Rest operator must be the LAST element.

✓ Correct

JavaScript

```
const [a, ...b] = arr;
```

27. What is Micro Stack and Macro Stack?

Microtask Queue and Macrotask Queue in JavaScript

(Interviewers sometimes incorrectly say “Micro Stack” and “Macro Stack”, but the correct terms are:)

✓ **Microtask Queue**

✓ **Macrotask Queue (Callback Queue)**

These are part of JavaScript’s asynchronous execution model and work with:

- Call Stack
- Web APIs
- Event Loop

Difference Between Rest and Spread

Both use:

JavaScript

...

But purpose differs.

Rest Operator

Collects values.

JavaScript

```
const [a, ...rest] = [1,2,3,4];
```

Spread Operator

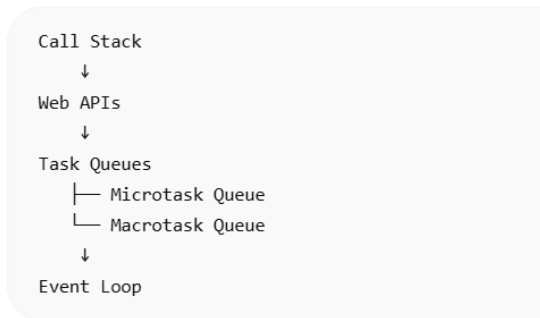
Expands values.

JavaScript

```
const arr = [1,2,3];

console.log(...arr);
```

JavaScript Execution Model



1. Macrotask Queue (Callback Queue)

Stores tasks like:

- `setTimeout`
- `setInterval`
- DOM events
- AJAX callbacks
- I/O operations

2. Microtask Queue

Microtasks have HIGHER priority than macrotasks.

Stores:

- Promise callbacks (`then`, `catch`, `finally`)
- `queueMicrotask()`
- `MutationObserver`

Example

```
</> JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

console.log("End");
```

Example

```
</> JavaScript
console.log("Start");

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output

Start
End
Timeout



Output

Start
End
Promise

Why Promise Executes First?

Because:

Microtask Queue has higher priority

Event loop processes:

- 1. Call Stack
- 2. All Microtasks
- 3. One Macrotask

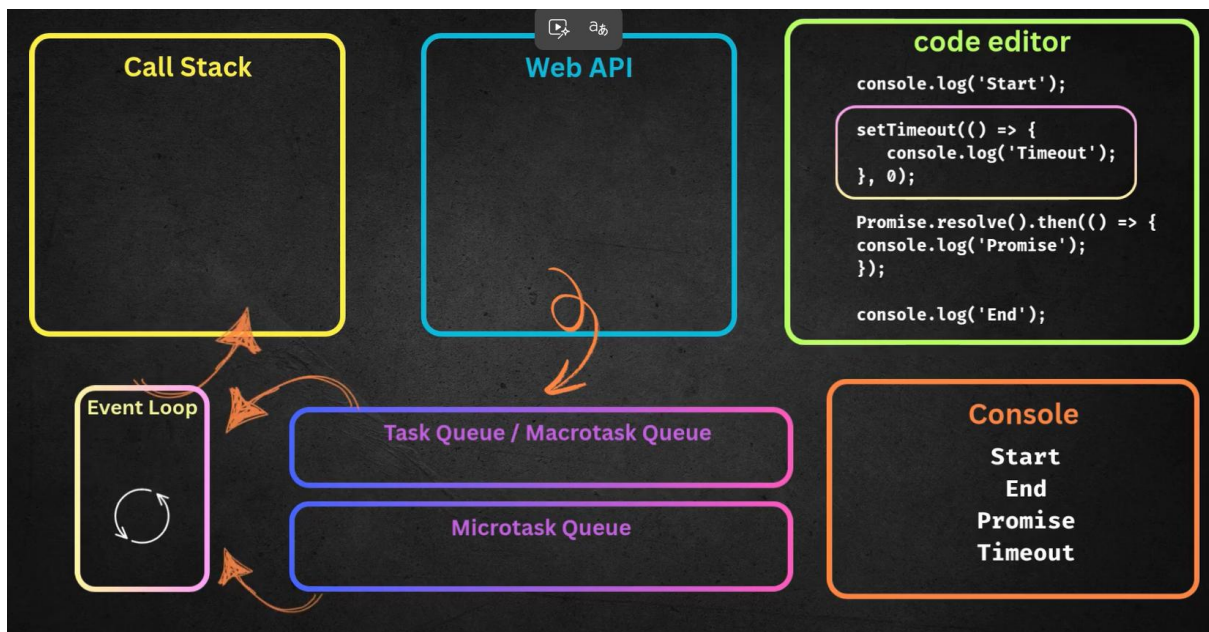
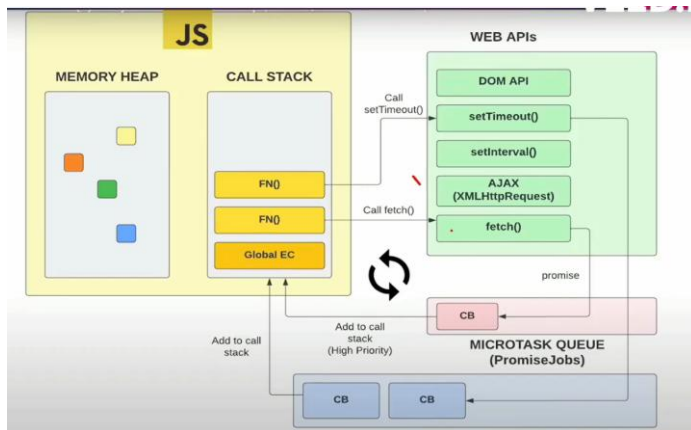
Execution Flow

Step-by-Step

- 1. Start → Call Stack
- 2. setTimeout → Web API
- 3. Promise.then → Microtask Queue
- 4. End → Call Stack
- 5. Promise executed
- 6. setTimeout executed

main.js	Output
<pre>1 console.log("Start"); 2 3 * setTimeout(function() { 4 console.log("Timeout");//macrotask/callback 5 }, 0); 6 7 * Promise.resolve().then(function() { 8 console.log("Promise"); // microTask! 9 }); 10</pre>	<pre>node /tmp/oy42E05F8C.js Start Promise Timeout</pre>

28. Call Stack Vs Event Loop => How is code executes in case of async code



Call Stack vs Event Loop in JavaScript

JavaScript is:

Single-threaded

It can execute only one task at a time using:

Call Stack

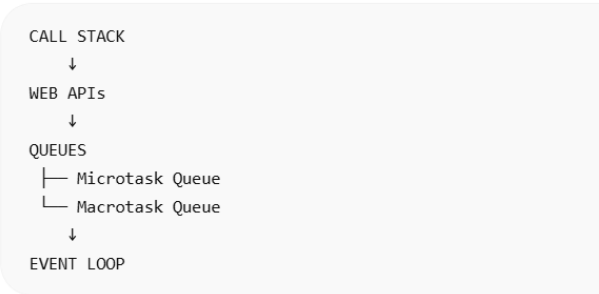
But JavaScript can still handle asynchronous operations because of:

- Web APIs
- Callback Queues
- Event Loop

Main Components

Component	Purpose
Call Stack	Executes functions
Web APIs	Handles async operations
Microtask Queue	Promise callbacks
Macrotask Queue	setTimeout callbacks
Event Loop	Moves tasks to stack

JavaScript Async Architecture



29. Higher Order functions

A **higher-order function (HOF)** is a function that does **at least one** of the following:

1. **Takes another function as an argument**, or
2. **Returns a function as a result**

JavaScript treats functions as **first-class citizens**, so functions can be stored in variables, passed as parameters, and returned from other functions.

1. Function Passed as Argument

Example: `forEach()`

```
</> JavaScript
let numbers = [1, 2, 3];

numbers.forEach(function(num) {
  console.log(num);
});
```

Explanation

- `forEach()` is a higher order function
- It accepts another function as parameter
- That function runs for every array item

Output

```
</> JavaScript
1
2
3
```

2. Function Returning Another Function

Example

```
</> JavaScript
function multiplyBy(x) {
  return function(y) {
    return x * y;
  };
}

let double = multiplyBy(2);

console.log(double(5));
```

Output

```
</> JavaScript
10
```

Explanation

- `multiplyBy(2)` returns another function
- Returned function multiplies value by `2`

Common Higher Order Functions in JavaScript

Function	Purpose
<code>map()</code>	Transform array
<code>filter()</code>	Filter values
<code>reduce()</code>	Reduce to single value
<code>forEach()</code>	Iterate array
<code>setTimeout()</code>	Executes callback later

30. Pure Functions and Impure Functions in JavaScript

A **Pure Function** is a function that:

1. Always returns the **same output** for the same input
2. Does **not modify external data**
3. Has **no side effects**

Characteristics of Pure Functions

- ✓ Same input → Same output
- ✓ No global variable modification

- ✓ No API calls / DOM changes
- ✓ No database/file modifications

Example of Pure Function

```
</> JavaScript

function add(a, b) {
  return a + b;
}

console.log(add(2, 3));
console.log(add(2, 3));
```

Output

```
</> JavaScript

5
5
```

Why Pure?

- Input (2,3) always gives 5
- No external variable used
- No side effects

Another Pure Function Example

```
</> JavaScript

function calculateTax(amount) {
  return amount * 0.18;
}

console.log(calculateTax(1000));
```

Output

```
</> JavaScript

180
```

Impure Functions in JavaScript

An **Impure Function** is a function that:

1. Depends on external data OR
2. Modifies external data OR
3. Produces side effects

Characteristics of Impure Functions

- ✗ Changes global variable
- ✗ API calls
- ✗ DOM updates
- ✗ Random values
- ✗ Date/time dependency

Example of Impure Function

```
</> JavaScript

let total = 0;

function add(value) {
  total += value;
  return total;
}

console.log(add(5));
console.log(add(5));
```

Output

```
</> JavaScript

5
10
```

Why Impure?

- Same input `5`
- Different outputs (`5`, `10`)
- Depends on external variable `total`

Another Impure Function Example

```
</> JavaScript

function showTime() {
  return new Date();
}

console.log(showTime());
```

Why Impure?

- Output changes every time
- Depends on system time

Pure vs Impure Function

Feature	Pure Function	Impure Function
Same output for same input	✓	✗
Uses external variables	✗	✓
Side effects	✗	✓
Easy to test	✓	✗
Predictable	✓	✗